GitHub Actions — Category 8: Monorepo Patterns

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–7 assumed. Monorepo tooling experience (npm workspaces, Go modules, Docker multi-stage) assumed. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers → Full YAML → Gotchas ⚠ → Connections

---

Table of Contents

- 8.1 Path Filters — paths and paths-ignore Triggers
- 8.2 Affected Service Detection — dorny/paths-filter, Custom Scripts
- 8.3 Dynamic Matrix from Changed Paths — Build Only What Changed
- 8.4 Monorepo CI/CD Design Patterns — Turborepo, Nx, Bazel Integration
- 8.5 Cross-Service Dependencies in Monorepos
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

8.1 Path Filters — paths and paths-ignore Triggers

What It Is

Path filters are conditions on `on:` triggers that restrict workflow execution to only occur when specific files or directories change. Without path filters, every push to any file in the repository triggers all workflows — expensive and noisy in a monorepo.

Jenkins equivalent: Multibranch pipelines with `when { changeset 'service-a/**' }` conditions, or SCM polling with include/exclude rules. The difference: GitHub's path filters are evaluated server-side before any runner is provisioned — a filtered-out push doesn't create a workflow run at all, not just a skipped one.

Why Path Filters Exist

```
Monorepo without path filters:
  Commit: fix typo in docs/README.md
  Result: ALL workflows trigger
    -> CI for service-a:      RUNS (irrelevant)
    -> CI for service-b:      RUNS (irrelevant)
    -> CI for service-c:      RUNS (irrelevant)
    -> Deploy pipeline:       RUNS (dangerous!)
    -> 20 minutes of wasted compute and noise
```

```
Monorepo WITH path filters:
  Commit: fix typo in docs/README.md
  Result:
    -> CI for service-a:      SKIPPED (no files in services/a/**)
    -> CI for service-b:      SKIPPED
    -> Deploy pipeline:       SKIPPED
    -> Docs workflow:         RUNS (docs/** matched)
    -> Total compute: ~2 minutes
```

How Path Filters Work Internally

```
Push event received by GitHub
        |
        v
GitHub evaluates path filter conditions
BEFORE dispatching to any runner
        |
        |-- paths: ['services/a/**'] matched?
        |     YES -> Create workflow run, dispatch to runner
        |     NO  -> Skip entirely (no run created, no UI entry)
        |
        v
For pull_request events:
  Compares PR head commit vs PR base branch
  (changed files in this PR)

For push events:
  Compares pushed commits vs previous commit
  (files changed in this push)

IMPORTANT: paths and paths-ignore use fnmatch glob patterns
  **  -> matches any path, including /
  *   -> matches any character except /
  ?   -> matches any single character
  []  -> character ranges: [abc], [0-9]
```

---

paths and paths-ignore — Complete Syntax Reference

```yaml
# FORM 1: paths -- workflow runs ONLY IF any listed path matches
on:
  push:
    paths:
      - 'services/payment/**'     # All files under services/payment/
      - 'shared/proto/**'         # Proto files (dependency of payment)
      - 'Dockerfile'              # Root Dockerfile
      - '*.go'                    # Any .go file at repo root
      - '**/*.go'                 # Any .go file anywhere in repo
```

```yaml
      - '.github/workflows/payment-ci.yml'  # The workflow itself
      # RULE: Always include the workflow file in its own paths list
      # If you don't, updating the workflow file doesn't re-run it

# FORM 2: paths-ignore -- workflow runs UNLESS ALL changed files match
on:
  push:
    paths-ignore:
      - '**/*.md'           # Ignore all markdown files
      - 'docs/**'           # Ignore docs directory
      - '.github/dependabot.yml'
      - 'LICENSE'
      - '.gitignore'
      - '**/*.txt'

# FORM 3: Branch + path combination
on:
  push:
    branches: [main, 'release/**']
    paths:
      - 'services/**'
      - 'shared/**'

# FORM 4: Per-event paths
on:
  push:
    branches: [main]
    paths:
      - 'services/payment/**'
  pull_request:
    branches: [main]
    paths:
      - 'services/payment/**'
  # Each event type can have independent path filters

# CANNOT combine paths and paths-ignore on the same event:
# on:
#   push:
#     paths: ['services/**']
#     paths-ignore: ['**/*.md']   # ERROR -- mutually exclusive
```

Per-Service Workflow Files (Simplest Monorepo Pattern)

```yaml
# .github/workflows/payment-service.yml
# Dedicated workflow per service with path filters

name: Payment Service CI

on:
  push:
```

```yaml
    branches: [main]
    paths:
      - 'services/payment/**'
      - 'shared/lib/**'           # Shared library used by payment
      - 'shared/proto/**'         # Proto definitions
      - '.github/workflows/payment-service.yml'
  pull_request:
    branches: [main]
    paths:
      - 'services/payment/**'
      - 'shared/lib/**'
      - 'shared/proto/**'
      - '.github/workflows/payment-service.yml'

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: services/payment
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2
      - uses: actions/setup-go@v5
        with:
          go-version-file: 'services/payment/go.mod'
          cache-dependency-path: 'services/payment/go.sum'
      - run: make test
      - run: make build

  docker-build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          context: .
          file: services/payment/Dockerfile
          push: ${{ github.ref == 'refs/heads/main' }}
          tags: ghcr.io/my-org/payment-service:sha-${{ github.sha }}
          cache-from: type=gha,scope=payment
          cache-to: type=gha,scope=payment,mode=max
```

The Branch Protection + Path Filter Problem

This is the single most important gotcha for monorepos:

```
Scenario:
  Branch protection requires status check: "Payment Service CI / test"
  Developer opens PR that only changes docs/architecture.md
  paths filter: ['services/payment/**'] does NOT match
  -> "Payment Service CI" workflow never runs
  -> No status check posted to the PR
  -> Branch protection BLOCKS merge: "Required status check pending"
  -> Developer is stuck

Three solutions:

SOLUTION A: Use paths-ignore instead of paths (preferred)
  on:
    pull_request:
      paths-ignore: ['**/*.md', 'docs/**']
  Workflow runs for all changes EXCEPT docs -> status check always posted

SOLUTION B: Global required check skip conditions
  Install "Skip CI" bot or use GitHub's "allow required check skip"
  When ALL changed files match paths-ignore, auto-pass the status check

SOLUTION C: Separate "trivial pass" job that always runs
  jobs:
    check-paths:
      runs-on: ubuntu-latest
      outputs:
        run-ci: ${{ steps.filter.outputs.payment }}
      steps:
        - uses: dorny/paths-filter@v3
          id: filter
          with:
            filters: |
              payment:
                - 'services/payment/**'

    test:
      needs: check-paths
      if: needs.check-paths.outputs.run-ci == 'true'
      runs-on: ubuntu-latest
      steps:
        - run: make test

    # This job ALWAYS runs and satisfies the branch protection check
    ci-gate:
      needs: [check-paths, test]
      if: always()
      runs-on: ubuntu-latest
      steps:
        - run: |
            if [ "${{ needs.test.result }}" = "success" ] || \
               [ "${{ needs.test.result }}" = "skipped" ]; then
              echo "CI gate: passed"
            else
```

```
                echo "CI gate: failed"
                exit 1
              fi
```

Interview Answer (30-45 seconds)

> "Path filters restrict workflow triggers to specific file patterns — evaluated server-side before any runner is provisioned. In a monorepo, you use `paths:` on per-service workflow files so only the relevant service's CI runs when that service changes. The critical design pattern: always include the workflow file itself in its paths list, otherwise changing the workflow doesn't re-run it. The major operational gotcha: path-filtered workflows that don't run leave no status check on the PR. If branch protection requires that status check, the PR is blocked. The fix is either use `paths-ignore` (workflow runs unless ALL changed files are docs/markdown), or add a final `ci-gate` job that always runs and satisfies the required check regardless of whether the actual test job was skipped."

Gotchas ⚠

- **`paths` and `paths-ignore` are mutually exclusive on the same trigger event.** You cannot use both simultaneously on `push:`. Choose one strategy and stick with it.
- **Path filters on `push:` compare against the PUSHED commit range**, not a base branch. For a push that bundles 3 commits, any file changed in any of those 3 commits triggers the filter. For `pull_request:`, it's the diff between the PR head and the base branch.
- **A workflow file change with no path match does NOT re-run the workflow.** If `payment-service.yml` has `paths: ['services/payment/**']` and you only change `payment-service.yml` itself, the workflow doesn't run — because the workflow file path isn't in the filter. Always add `.github/workflows/your-workflow.yml` to your own paths list.
- **Nested `**` patterns work differently than expected.** `services/**` matches `services/payment/src/handler.go` (any depth). `services/*` matches only `services/payment` (one level only). Use `**` for recursive matching.
- **`paths-ignore` logic is "skip if ALL changed files match"**, not "skip if ANY changed file matches." If a PR changes `docs/README.md` AND `services/payment/handler.go`, and `paths-ignore` includes `docs/**`, the workflow STILL runs because `services/payment/handler.go` does not match the ignore pattern.

8.2 Affected Service Detection — dorny/paths-filter, Custom Scripts ⚠

What It Is

Affected service detection is the runtime determination of which services, packages, or components were changed by a commit or PR — producing a structured output that drives downstream job execution. Unlike

static path filters (which gate the entire workflow), affected service detection produces a data structure (list of services) that parameterizes what runs inside the workflow.

Jenkins equivalent: Jenkins with the `changeset` condition in `when {}` blocks, or scripted pipeline loops over `currentBuild.changeSets`. The difference: GitHub Actions produces this output as job outputs (JSON arrays) that feed directly into matrix strategies — Jenkins has no equivalent first-class mechanism.

Why Detection Is a Separate Concern from Filtering

```
Path filters (8.1):    Course-grained. "Should this WORKFLOW run at all?"
Service detection (8.2): Fine-grained. "WHICH SERVICES need to be built?"

Example:
  Monorepo has: auth, payment, inventory, orders, notifications (5
services)
  PR changes: services/payment/handler.go AND shared/proto/api.proto

  Path filter on workflow:
    on:
      push:
        paths: ['services/**', 'shared/**']
    -> YES, workflow should run (shared/proto changed)

  Service detection inside the workflow:
    -> payment: AFFECTED (directly changed)
    -> auth:    AFFECTED (imports shared/proto)
    -> inventory: AFFECTED (imports shared/proto)
    -> orders:  NOT affected
    -> notifications: NOT affected
    -> Run matrix for: [payment, auth, inventory]
```

Method 1: dorny/paths-filter (Most Common)

`dorny/paths-filter` is a popular action that evaluates a set of named path patterns against the changed files and outputs boolean flags and lists per pattern.

```yaml
name: Monorepo CI with paths-filter

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  detect-changes:
    runs-on: ubuntu-latest
```

```yaml
    outputs:
      # Boolean flags per service
      auth:         ${{ steps.filter.outputs.auth }}
      payment:      ${{ steps.filter.outputs.payment }}
      inventory:    ${{ steps.filter.outputs.inventory }}
      orders:       ${{ steps.filter.outputs.orders }}
      # JSON array of changed services (for matrix)
      changed-services: ${{ steps.filter.outputs.changes }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  # v4.2.2

      - uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36 # v3.0.2
        id: filter
        with:
          # For PRs: compares head vs base branch
          # For pushes: compares pushed commits vs previous state
          list-files: shell    # Output format for changed file lists
          filters: |
            auth:
              - 'services/auth/**'
              - 'shared/lib/**'           # Shared dep used by auth
            payment:
              - 'services/payment/**'
              - 'shared/lib/**'
              - 'shared/proto/**'         # Payment uses protos
            inventory:
              - 'services/inventory/**'
              - 'shared/lib/**'
            orders:
              - 'services/orders/**'
              - 'shared/lib/**'
              - 'shared/proto/**'

  # Use boolean outputs for individual service jobs
  test-auth:
    needs: detect-changes
    if: needs.detect-changes.outputs.auth == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make test SERVICE=auth

  test-payment:
    needs: detect-changes
    if: needs.detect-changes.outputs.payment == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make test SERVICE=payment

  # Use JSON array output for matrix
  test-matrix:
```

```yaml
    needs: detect-changes
    if: needs.detect-changes.outputs.changes != '[]'
    strategy:
      fail-fast: false
      matrix:
        service: ${{ fromJSON(needs.detect-changes.outputs.changes) }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make test SERVICE=${{ matrix.service }}
```

---

paths-filter Advanced Configuration

```yaml
- uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
  id: filter
  with:
    # TOKEN required for PRs from forks (to fetch PR diff)
    token: ${{ github.token }}

    # Output formats:
    # csv:   comma-separated list of changed files per filter
    # json:  JSON array of changed files per filter
    # shell: shell array (space-separated, escaped)
    # none:  only output boolean (default)
    list-files: json

    filters: |
      # Simple directory match
      backend:
        - 'backend/**'

      # Multiple patterns for one filter (OR logic)
      frontend:
        - 'frontend/**'
        - 'packages/ui/**'
        - 'packages/icons/**'

      # Exclude pattern (! prefix)
      backend-non-test:
        - 'backend/**'
        - '!backend/**/*.test.ts'
        - '!backend/**/__tests__/**'

      # Match specific file types across the repo
      go-files:
        - '**/*.go'
        - '!vendor/**'

      # Infrastructure changes
      infrastructure:
```

```
          - 'terraform/**'
          - 'k8s/**'
          - '.github/workflows/**'

        # Shared dependencies -- if changed, everything is affected
        shared-critical:
          - 'packages/shared-types/**'
          - 'packages/api-client/**'

outputs in steps:
  steps.filter.outputs.backend        -> 'true' or 'false'
  steps.filter.outputs.backend_files  -> JSON array of changed files in
backend
  steps.filter.outputs.changes        -> JSON array of filter names that
matched
                                          e.g.,
'["backend","infrastructure"]'
```

Method 2: Custom git diff Script (Maximum Control)

```
# Custom detection with full control over logic
# Handles: shared dependency graph, monorepo topology, per-service config

jobs:
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      services: ${{ steps.detect.outputs.services }}
      has-shared-changes: ${{ steps.detect.outputs.has-shared }}
      infra-changed: ${{ steps.detect.outputs.infra }}
      matrix: ${{ steps.detect.outputs.matrix }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0    # Full history for reliable git diff

      - name: Detect changed services
        id: detect
        run: |
          set -euo pipefail

          # Determine diff base
          if [ "${{ github.event_name }}" = "pull_request" ]; then
            # For PRs: diff between PR head and base branch
            BASE="${{ github.event.pull_request.base.sha }}"
            HEAD="${{ github.event.pull_request.head.sha }}"
          elif [ "${{ github.event_name }}" = "push" ]; then
            # For pushes: diff between before and after
            BASE="${{ github.event.before }}"
            HEAD="${{ github.sha }}"
```

```
          else
            # Fallback: compare with HEAD~1
            BASE="HEAD~1"
            HEAD="HEAD"
          fi

          echo "Comparing: $BASE -> $HEAD"
          CHANGED_FILES=$(git diff --name-only "$BASE" "$HEAD" 2>/dev/null
|| \
                          git diff --name-only HEAD~1 HEAD)

          echo "=== Changed files ===="
          echo "$CHANGED_FILES"
          echo "====================="

          # ── Detect shared changes ──────────────────────────────────
          HAS_SHARED="false"
          INFRA_CHANGED="false"

          if echo "$CHANGED_FILES" | grep -qE
'^(packages/shared|packages/api-client|packages/types)/'; then
            HAS_SHARED="true"
            echo "SHARED changes detected -- all services affected"
          fi

          if echo "$CHANGED_FILES" | grep -qE
'^(terraform|k8s|\.github)/'; then
            INFRA_CHANGED="true"
            echo "Infrastructure changes detected"
          fi

          # ── Detect changed services ────────────────────────────────
          # Map: service name -> path prefix
          declare -A SERVICE_PATHS=(
            ["auth"]="services/auth"
            ["payment"]="services/payment"
            ["inventory"]="services/inventory"
            ["orders"]="services/orders"
            ["notifications"]="services/notifications"
            ["api-gateway"]="services/api-gateway"
          )

          # Map: service name -> shared dependencies it uses
          # If a shared dep changes, all services that use it are affected
          declare -A SERVICE_SHARED_DEPS=(
            ["auth"]="packages/shared packages/types"
            ["payment"]="packages/shared packages/proto packages/api-
client"
            ["inventory"]="packages/shared packages/types"
            ["orders"]="packages/shared packages/proto packages/api-
client"
            ["notifications"]="packages/shared"
            ["api-gateway"]="packages/shared packages/api-client"
          )
```

```
          AFFECTED_SERVICES=()

          for SERVICE in "${!SERVICE_PATHS[@]}"; do
            PATH_PREFIX="${SERVICE_PATHS[$SERVICE]}"
            DEPS="${SERVICE_SHARED_DEPS[$SERVICE]}"

            # Check: direct file changes in service path?
            DIRECT_CHANGE=$(echo "$CHANGED_FILES" | \
              grep -c "^${PATH_PREFIX}/" || true)

            # Check: any of this service's shared deps changed?
            DEP_CHANGE=0
            for DEP in $DEPS; do
              COUNT=$(echo "$CHANGED_FILES" | grep -c "^${DEP}/" || true)
              DEP_CHANGE=$((DEP_CHANGE + COUNT))
            done

            if [ "$DIRECT_CHANGE" -gt 0 ] || [ "$DEP_CHANGE" -gt 0 ] || [
"$HAS_SHARED" = "true" ]; then
              AFFECTED_SERVICES+=("$SERVICE")
              echo "AFFECTED: $SERVICE (direct: $DIRECT_CHANGE, dep:
$DEP_CHANGE)"
            else
              echo "NOT affected: $SERVICE"
            fi
          done

          # ── Build JSON outputs ──────────────────────────────────────
          if [ ${#AFFECTED_SERVICES[@]} -eq 0 ]; then
            SERVICES_JSON="[]"
            MATRIX_JSON='{"include":[]}'
          else
            # Simple array for matrix
            SERVICES_JSON=$(printf '%s\n' "${AFFECTED_SERVICES[@]}" | \
              jq -R . | jq -s -c .)

            # Rich matrix with service-specific metadata from config files
            INCLUDES='[]'
            for SERVICE in "${AFFECTED_SERVICES[@]}"; do
              CONFIG_FILE="services/${SERVICE}/.service-config.json"
              if [ -f "$CONFIG_FILE" ]; then
                SERVICE_CONFIG=$(cat "$CONFIG_FILE")
              else
                # Default config
                SERVICE_CONFIG=$(jq -n \
                  --arg name "$SERVICE" \
                  '{name: $name, language: "unknown", deploy: true}')
              fi
              INCLUDES=$(echo "$INCLUDES" | jq \
                --argjson config "$SERVICE_CONFIG" \
                ". + [$config]")
            done
            MATRIX_JSON=$(jq -n --argjson includes "$INCLUDES"
```

```
  '{"include": $includes}')
          fi

          echo "Affected services: $SERVICES_JSON"
          echo "Matrix: $MATRIX_JSON"

          # Write outputs
          echo "services=${SERVICES_JSON}" >> $GITHUB_OUTPUT
          echo "has-shared=${HAS_SHARED}" >> $GITHUB_OUTPUT
          echo "infra=${INFRA_CHANGED}" >> $GITHUB_OUTPUT
          echo "matrix=${MATRIX_JSON}" >> $GITHUB_OUTPUT
```

Method 3: Python-Based Detection (Complex Dependency Graphs)

```
      - name: Advanced dependency-aware detection
        id: detect-python
        run: |
          python3 << 'PYEOF'
          import subprocess
          import json
          import os
          import sys
          from pathlib import Path

          # Get changed files
          event = os.environ.get('GITHUB_EVENT_NAME')
          if event == 'pull_request':
              base = os.environ['BASE_SHA']
              head = os.environ['HEAD_SHA']
          else:
              base = subprocess.run(
                  ['git', 'rev-parse', 'HEAD~1'],
                  capture_output=True, text=True
              ).stdout.strip()
              head = 'HEAD'

          result = subprocess.run(
              ['git', 'diff', '--name-only', base, head],
              capture_output=True, text=True
          )
          changed_files = set(result.stdout.strip().split('\n'))
          print(f"Changed files: {len(changed_files)}")

          # Load dependency graph from manifest
          with open('services/dependency-graph.json') as f:
              dep_graph = json.load(f)
          # Format:
          # {
          #   "payment": {
          #     "path": "services/payment",
```

```
            #     "deps": ["packages/proto", "packages/shared"],
            #     "language": "go",
            #     "deploy_env": "production"
            #   },
            #   ...
            # }

            def is_affected(service_name, service_config, changed):
                """Check if a service is affected by the changed files."""
                service_path = service_config['path']

                # Direct change
                if any(f.startswith(service_path + '/') for f in changed):
                    return True, 'direct'

                # Dependency change
                for dep in service_config.get('deps', []):
                    if any(f.startswith(dep + '/') for f in changed):
                        return True, f'dep:{dep}'

                return False, None

            # Compute transitively affected services
            affected = {}
            for service_name, service_config in dep_graph.items():
                is_aff, reason = is_affected(service_name, service_config,
  changed_files)
                if is_aff:
                    affected[service_name] = {**service_config, 'reason':
  reason}

            # Build matrix
            matrix_items = []
            for name, config in affected.items():
                matrix_items.append({
                    'service': name,
                    'path': config['path'],
                    'language': config.get('language', 'unknown'),
                    'deploy_env': config.get('deploy_env', ''),
                    'reason': config.get('reason', 'unknown')
                })

            matrix = {'include': matrix_items}
            services = [item['service'] for item in matrix_items]

            print(f"Affected services: {services}")
            print(f"Matrix: {json.dumps(matrix, indent=2)}")

            # Write to GITHUB_OUTPUT
            with open(os.environ['GITHUB_OUTPUT'], 'a') as f:
                f.write(f"services={json.dumps(services)}\n")
                f.write(f"matrix={json.dumps(matrix, separators=(',',
  ':'))}\n")
                f.write(f"has-changes={'true' if services else 'false'}\n")
```

```
          PYEOF
        env:
          BASE_SHA: ${{ github.event.pull_request.base.sha || 
github.event.before }}
          HEAD_SHA: ${{ github.event.pull_request.head.sha || github.sha 
}}
```

Interview Answer (30-45 seconds)

> "Affected service detection runs inside the workflow to determine which services need to build and test, as opposed to path filters which decide if the workflow runs at all. The standard approach uses `dorny/paths-filter`: you declare named patterns per service, the action evaluates which patterns match the changed files, and outputs both boolean flags (for conditional individual jobs) and a JSON array called `changes` (for matrix strategy). The output `changes` contains only the filter names that matched — e.g., `[\"payment\", \"auth\"]` — and `fromJSON()` feeds this directly into a matrix. For complex dependency graphs, a custom script with `git diff --name-only` gives full control: you compute direct changes AND transitive dependency changes. The critical input for accuracy: `fetch-depth: 0` on checkout so git has full history for reliable diff computation."

Gotchas ⚠

- **`fetch-depth: 0` is required for reliable diffs.** The default `fetch-depth: 1` (shallow clone) means `git diff` may not have the base commit available, causing the diff command to fail or fall back to comparing against HEAD~1 only, which is wrong for multi-commit pushes.
- **`github.event.before` is all zeros (`0000000000...`) on the first push to a new branch.** `git diff 0000... HEAD` fails. Always handle this case: if `before` is all zeros, either skip detection (no previous state) or compare against HEAD~1.
- **`dorny/paths-filter` outputs `changes` as a JSON array of filter NAMES, not file paths.** `steps.filter.outputs.changes` is `'["auth","payment"]'`, not the list of changed files. The changed file list is `steps.filter.outputs.auth_files` (if `list-files: json`).
- **Force pushes break git diff.** If someone force-pushes a branch, `github.event.before` is the SHA that was overwritten. `git diff before after` shows all changes since the force push rebase point, which may be a very large diff. Handle gracefully by checking if the before SHA exists: `git cat-file -e $BASE_SHA || echo "SHA not found"`.
- **Merge commits in push events include ALL changes from the merged branch.** On a squash-and-merge or merge commit, the diff includes every file changed across the entire PR. This is usually desired but can produce unexpectedly large affected service lists.

8.3 Dynamic Matrix from Changed Paths — Build Only What Changed

---

What It Is

The complete pattern combining service detection (8.2) with matrix strategy (Cat 5.1) to produce parallel, targeted CI execution — running tests and builds only for the services that were actually affected by a commit.

This topic is where the full power of GitHub Actions' flexibility is realized. It is frequently tested in senior interviews because it requires understanding job outputs, `fromJSON()`, matrix strategy, and service detection simultaneously.

The Full Two-Stage Pattern

```
Stage 1 (detect-changes job):
  - Run git diff or paths-filter
  - Produce JSON array of affected services
  - Output as job output

Stage 2 (build-and-test job with matrix):
  - needs: detect-changes
  - strategy.matrix: ${{ fromJSON(needs.detect-changes.outputs.matrix) }}
  - One parallel job per affected service
```

Complete Dynamic Matrix Implementation

```yaml
name: Monorepo Dynamic Matrix CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:

  # ── Stage 1: Detect what changed ──────────────────────────────
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.build-matrix.outputs.matrix }}
      has-changes: ${{ steps.build-matrix.outputs.has-changes }}
      infra-changed: ${{ steps.build-matrix.outputs.infra-changed }}
      shared-changed: ${{ steps.build-matrix.outputs.shared-changed }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0

      - name: Detect changes with paths-filter
        id: filter
```

```
        uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
# v3.0.2
        with:
          list-files: none
          filters: |
            auth:
              - 'services/auth/**'
              - 'packages/shared/**'
              - 'packages/types/**'
            payment:
              - 'services/payment/**'
              - 'packages/shared/**'
              - 'packages/proto/**'
            inventory:
              - 'services/inventory/**'
              - 'packages/shared/**'
              - 'packages/types/**'
            orders:
              - 'services/orders/**'
              - 'packages/shared/**'
              - 'packages/proto/**'
            frontend:
              - 'apps/web/**'
              - 'packages/ui/**'
              - 'packages/icons/**'
            infrastructure:
              - 'terraform/**'
              - 'k8s/**'
              - '.github/workflows/**'
            shared:
              - 'packages/shared/**'
              - 'packages/types/**'
              - 'packages/proto/**'

      - name: Build typed matrix
        id: build-matrix
        run: |
          # Service metadata -- what to do with each service when it
changes
          # Language determines test command; size determines runner
          declare -A SERVICE_LANGUAGE=(
            [auth]="go"
            [payment]="go"
            [inventory]="python"
            [orders]="go"
            [frontend]="node"
          )
          declare -A SERVICE_RUNNER=(
            [auth]="ubuntu-latest"
            [payment]="ubuntu-latest"
            [inventory]="ubuntu-latest"
            [orders]="ubuntu-latest"
            [frontend]="ubuntu-8-cores"   # Frontend needs more RAM
          )
```

```
            declare -A SERVICE_DEPLOY_ENV=(
              [auth]="production"
              [payment]="production"
              [inventory]="production"
              [orders]="production"
              [frontend]="production"
            )
            declare -A SERVICE_TIMEOUT=(
              [auth]="10"
              [payment]="15"
              [inventory]="8"
              [orders]="10"
              [frontend]="20"
            )

            CHANGES='${{ steps.filter.outputs.changes }}'
            echo "Changed filter groups: $CHANGES"

            # Build matrix include entries
            INCLUDES="[]"
            SERVICES=()

            # Parse the changes JSON array
            CHANGED_SERVICES=$(echo "$CHANGES" | jq -r '.[]' | \
              grep -v "infrastructure" | grep -v "shared" || true)

            for SERVICE in $CHANGED_SERVICES; do
              if [ -z "${SERVICE_LANGUAGE[$SERVICE]+x}" ]; then
                echo "Warning: Unknown service $SERVICE, skipping"
                continue
              fi

              ENTRY=$(jq -n \
                --arg service "$SERVICE" \
                --arg lang "${SERVICE_LANGUAGE[$SERVICE]}" \
                --arg runner "${SERVICE_RUNNER[$SERVICE]}" \
                --arg deploy_env "${SERVICE_DEPLOY_ENV[$SERVICE]}" \
                --argjson timeout "${SERVICE_TIMEOUT[$SERVICE]}" \
                '{
                  service: $service,
                  language: $lang,
                  runner: $runner,
                  deploy_env: $deploy_env,
                  timeout_minutes: $timeout
                }')
              INCLUDES=$(echo "$INCLUDES" | jq --argjson entry "$ENTRY" '. +
[$entry]')
              SERVICES+=("$SERVICE")
            done

            HAS_CHANGES="false"
            if [ ${#SERVICES[@]} -gt 0 ]; then
              HAS_CHANGES="true"
            fi
```

```
          MATRIX=$(jq -n --argjson includes "$INCLUDES" '{"include":
$includes}')
          SHARED="${{ steps.filter.outputs.shared }}"
          INFRA="${{ steps.filter.outputs.infrastructure }}"

          echo "Final matrix: $MATRIX"
          echo "matrix=${MATRIX}" >> $GITHUB_OUTPUT
          echo "has-changes=${HAS_CHANGES}" >> $GITHUB_OUTPUT
          echo "infra-changed=${INFRA}" >> $GITHUB_OUTPUT
          echo "shared-changed=${SHARED}" >> $GITHUB_OUTPUT

  # ── Stage 2: Build and test affected services in parallel ────────
  build-and-test:
    needs: detect-changes
    if: needs.detect-changes.outputs.has-changes == 'true'
    strategy:
      fail-fast: false    # Don't cancel siblings if one fails
      matrix: ${{ fromJSON(needs.detect-changes.outputs.matrix) }}
    # Use per-service runner specification from the matrix
    runs-on: ${{ matrix.runner }}
    timeout-minutes: ${{ fromJSON(matrix.timeout_minutes) }}
    name: "${{ matrix.service }} (${{ matrix.language }})"

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      # Language-specific setup
      - name: Setup Go
        if: matrix.language == 'go'
        uses: actions/setup-go@v5
        with:
          go-version-file: 'services/${{ matrix.service }}/go.mod'
          cache-dependency-path: 'services/${{ matrix.service }}/go.sum'

      - name: Setup Python
        if: matrix.language == 'python'
        uses: actions/setup-python@v5
        with:
          python-version-file: 'services/${{ matrix.service }}/.python-
version'
          cache: 'pip'
          cache-dependency-path: 'services/${{ matrix.service
}}/requirements*.txt'

      - name: Setup Node
        if: matrix.language == 'node'
        uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version-file: 'services/${{ matrix.service }}/.nvmrc'
          cache: 'npm'
          cache-dependency-path: 'services/${{ matrix.service }}/package-
lock.json'
```

```yaml
      # Restore service-specific build cache
      - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8  #
v4.1.1
        with:
          path: services/${{ matrix.service }}/.build-cache
          key: ${{ runner.os }}-${{ matrix.service }}-${{ matrix.language
}}-${{ hashFiles(format('services/{0}/**', matrix.service)) }}
          restore-keys: |
            ${{ runner.os }}-${{ matrix.service }}-${{ matrix.language }}-

      # Lint
      - name: Lint
        working-directory: services/${{ matrix.service }}
        run: make lint

      # Test with coverage
      - name: Test
        id: test
        working-directory: services/${{ matrix.service }}
        run: make test-coverage
        env:
          TEST_DATABASE_URL: ${{ secrets.TEST_DATABASE_URL }}

      # Upload coverage
      - name: Upload coverage
        uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: coverage-${{ matrix.service }}
          path: services/${{ matrix.service }}/coverage/
          retention-days: 7

      # Build artifact
      - name: Build
        working-directory: services/${{ matrix.service }}
        run: make build

      # Docker build + push for non-PR runs
      - name: Docker build and push
        if: github.event_name == 'push'
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          context: .
          file: services/${{ matrix.service }}/Dockerfile
          push: true
          tags: ghcr.io/my-org/${{ matrix.service }}:sha-${{ github.sha }}
          cache-from: type=gha,scope=${{ matrix.service }}
          cache-to: type=gha,scope=${{ matrix.service }},mode=max

  # ── Stage 3: Deploy affected services ──────────────────────────
  deploy:
    needs: [detect-changes, build-and-test]
    if: |
      github.event_name == 'push' &&
```

```
      github.ref == 'refs/heads/main' &&
      needs.detect-changes.outputs.has-changes == 'true'
    strategy:
      fail-fast: false
      matrix: ${{ fromJSON(needs.detect-changes.outputs.matrix) }}
    runs-on: ubuntu-latest
    environment: ${{ matrix.deploy_env }}
    name: "deploy ${{ matrix.service }}"
    concurrency:
      group: deploy-${{ matrix.service }}-${{ matrix.deploy_env }}
      cancel-in-progress: false    # Never cancel an in-progress deploy

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
          aws-region: us-east-1
      - run: |
          ./deploy.sh \
            --service ${{ matrix.service }} \
            --image ghcr.io/my-org/${{ matrix.service }}:sha-${{
github.sha }} \
            --environment ${{ matrix.deploy_env }}

  # ── Stage 4: Infrastructure (if infra changed) ──────────────────
  terraform:
    needs: detect-changes
    if: needs.detect-changes.outputs.infra-changed == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: echo "Running Terraform plan..."

  # ── Stage 5: Gate job that always posts a status check ──────────
  ci-gate:
    needs: [detect-changes, build-and-test, deploy]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - name: Evaluate gate
        run: |
          echo "detect-changes: ${{ needs.detect-changes.result }}"
          echo "build-and-test: ${{ needs.build-and-test.result }}"
          echo "deploy:         ${{ needs.deploy.result }}"

          # Fail if any required job failed
          # (skipped is OK -- means no changes for that job)
          for RESULT in \
            "${{ needs.detect-changes.result }}" \
            "${{ needs.build-and-test.result }}" \
            "${{ needs.deploy.result }}"; do
            if [ "$RESULT" = "failure" ] || [ "$RESULT" = "cancelled" ];
```

```
then
              echo "Gate FAILED: a required job failed or was cancelled"
              exit 1
            fi
          done
          echo "Gate PASSED: all jobs succeeded or were appropriately
skipped"
```

Collecting and Aggregating Matrix Results

```
# Fan-out/fan-in pattern: collect all service test results

  # Fan-out: matrix runs per service (as above)

  # Fan-in: aggregate results after all matrix jobs complete
  aggregate-results:
    needs: build-and-test
    if: always()
    runs-on: ubuntu-latest
    steps:
      # Download all coverage artifacts
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          pattern: coverage-*
          merge-multiple: true
          path: all-coverage/

      - name: Generate combined coverage report
        run: |
          # Combine all coverage files into a single report
          # Tool depends on language -- example for Go
          gocovmerge all-coverage/**/*.out > combined.out
          go tool cover -html=combined.out -o coverage-report.html
          go tool cover -func=combined.out | tail -1

      - name: Post coverage comment on PR
        if: github.event_name == 'pull_request'
        uses: actions/github-
script@60a0d83039c74a4aee543508d2ffcb1c3799cdea
        with:
          script: |
            const coverage = require('fs')
              .readFileSync('coverage-summary.txt', 'utf8');
            github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
              body: `## Coverage Report\n\`\`\`\n${coverage}\n\`\`\``
            });
```

```yaml
      - name: Upload combined report
        uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: combined-coverage-report
          path: coverage-report.html
```

Interview Answer (30-45 seconds)

> "The dynamic matrix pattern combines service detection with GitHub's matrix strategy. A first job runs git diff or paths-filter to produce a JSON array of affected services, outputting it with `echo 'matrix=...' >> $GITHUB_OUTPUT`. A second job reads `needs.detect-changes.outputs.matrix` and passes it through `fromJSON()` to `strategy.matrix`. GitHub expands this into parallel jobs — one per affected service. The matrix entries can be rich objects, not just strings: each entry can carry the language, runner type, deployment environment, and timeout for that service, making the downstream job fully parameterized without hardcoding anything. Three critical details: `fetch-depth: 0` for reliable git history, an empty matrix guard with `if: needs.detect.outputs.has-changes == 'true'` to prevent the 'matrix vector is empty' error, and `fail-fast: false` so one failing service doesn't cancel all others."

Gotchas ⚠

- **Empty matrix (`{"include":[]}`) causes a workflow error.** Always check before feeding to matrix. The guard `if: needs.detect.outputs.has-changes == 'true'` prevents the downstream job from running when there are no affected services.
- **`fromJSON()` on a matrix returns an OBJECT, not an array, for complex matrices.** When the matrix is `{"include":[...]}`, `strategy.matrix` should be set to the full object. When it's a simple array like `["auth","payment"]`, you need `{service: ${{ fromJSON(...) }}}` wrapping.
- **Matrix job names must be unique within a workflow.** If two matrix jobs produce the same `name:` (e.g., from a non-unique combination), GitHub will deduplicate them unpredictably. Always include the matrix variable in the job name.
- **`runs-on: ${{ matrix.runner }}` is evaluated as an expression.** If `matrix.runner` contains a value that doesn't match any available runner, the job fails with "no runner" rather than a helpful error. Validate runner label values in the detect job.
- **Concurrency on matrix deploy jobs requires service-scoped groups.** Using a single group for all matrix deploy jobs serializes ALL deploys regardless of service. Use `group: deploy-${{ matrix.service }}-${{ matrix.deploy_env }}` to allow parallel deploys of different services while serializing multiple deploys of the same service.

8.4 Monorepo CI/CD Design Patterns — Turborepo, Nx, Bazel Integration

---

What It Is

Build orchestration tools designed specifically for monorepos — they understand your package dependency graph, cache build outputs, and determine which packages need to rebuild when something changes. GitHub Actions integrates with these tools to make CI faster and smarter.

The Core Problem These Tools Solve

```
Naive monorepo CI (without build tools):
  30-package monorepo
  PR changes 1 package
  GitHub Actions detects change (via dynamic matrix) -> runs CI for 1
package
  BUT: 5 other packages depend on the changed package -> also need
rebuilding
  AND: building package-a requires building its deps first -> ordering
matters
  AND: if we run the same tests again with no code change -> wasted
compute

Build tool solution:
  Tool knows the dependency graph
  Tool has a content-addressed cache
  Tool runs only affected tasks in the correct order
  Tool skips tasks whose inputs haven't changed (cache hit)
```

Turborepo Integration

Turborepo is a Rust-powered build system for JavaScript/TypeScript monorepos. It caches task outputs based on input hashes and can distribute cache remotely.

```
# turbo.json (in repo root)
# {
#   "$schema": "https://turbo.build/schema.json",
#   "tasks": {
#     "build": {
#       "dependsOn": ["^build"],    // Build deps first
#       "outputs": ["dist/**", ".next/**"]
#     },
#     "test": {
#       "dependsOn": ["^build"],    // Test needs deps built
#       "outputs": ["coverage/**"]
#     },
#     "lint": {
#       "outputs": []
#     },
#     "type-check": {
```

```
#       "dependsOn": ["^build"],
#       "outputs": []
#     }
#   }
# }

name: Turborepo CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  packages: read

env:
  # Turborepo remote cache (Vercel or self-hosted)
  TURBO_TOKEN: ${{ secrets.TURBO_TOKEN }}
  TURBO_TEAM: ${{ vars.TURBO_TEAM }}
  TURBO_REMOTE_ONLY: false

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0    # Turbo uses git to determine affected
packages

      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: '20'
          cache: 'npm'

      - run: npm ci

      # Turborepo local cache backed by GitHub Actions cache
      - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8  #
v4.1.1
        with:
          path: .turbo
          key: ${{ runner.os }}-turbo-${{ github.sha }}
          restore-keys: |
            ${{ runner.os }}-turbo-

      # Run lint, type-check, and test for ALL affected packages
      # Turbo determines "affected" via git diff + dependency graph
      - name: Lint affected packages
        run: npx turbo lint --filter='...[origin/main]'
        # --filter='...[origin/main]' = packages changed since main
```

```yaml
          # + their dependents (because they might be affected)

        - name: Type check affected packages
          run: npx turbo type-check --filter='...[origin/main]'

        - name: Test affected packages
          run: |
            npx turbo test --filter='...[origin/main]' \
              -- --coverage --ci

        - name: Build affected packages
          run: npx turbo build --filter='...[origin/main]'

        # Upload coverage for all packages
        - uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
          with:
            name: coverage-reports
            path: '**/coverage/'
            retention-days: 7

    # Docker builds for services that changed
    docker-builds:
      needs: ci
      if: github.event_name == 'push'
      runs-on: ubuntu-latest
      steps:
        - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
          with:
            fetch-depth: 0

        - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
          with:
            node-version: '20'
            cache: 'npm'

        - run: npm ci

        - name: Get changed deployable services
          id: services
          run: |
            # Use turbo to identify which apps changed
            CHANGED=$(npx turbo run build \
              --filter='...[origin/main]' \
              --dry-run=json \
              | jq -c '[.packages[] | select(startswith("apps/")) |
ltrimstr("apps/")]')
            echo "services=${CHANGED}" >> $GITHUB_OUTPUT

        - uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d
        - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
          with:
            registry: ghcr.io
            username: ${{ github.actor }}
```

```
          password: ${{ secrets.GITHUB_TOKEN }}
        permissions:
          packages: write

      - name: Build and push changed service images
        run: |
          SERVICES='${{ steps.services.outputs.services }}'
          echo "$SERVICES" | jq -r '.[]' | while read SERVICE; do
            echo "Building: $SERVICE"
            docker buildx build \
              --file apps/$SERVICE/Dockerfile \
              --tag ghcr.io/my-org/$SERVICE:sha-${{ github.sha }} \
              --push \
              --cache-from type=gha,scope=$SERVICE \
              --cache-to type=gha,scope=$SERVICE,mode=max \
              .
          done
```

Nx Integration

Nx is a build system for polyglot monorepos (JavaScript, TypeScript, Java, Python, Go, etc.) with a sophisticated project graph and distributed task execution.

```
name: Nx CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # Nx Cloud distributed task execution (optional but powerful)
  # Splits tasks across multiple agents
  main:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0    # Required for nx affected computation

      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: '20'
          cache: 'npm'

      - run: npm ci

      # Set base and head for affected calculation
      - name: Set SHAs for nx affected
```

```yaml
        uses: nrwl/nx-set-shas@v4
        # Sets: NX_BASE and NX_HEAD environment variables
        # NX_BASE = last successful CI run SHA (not just HEAD~1!)
        # This ensures affected is computed correctly for multi-commit
pushes

      # Nx local cache
      - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
        with:
          path: |
            ~/.nx/cache
            .nx/cache
          key: ${{ runner.os }}-nx-${{ hashFiles('**/package-lock.json')
}}-${{ github.sha }}
          restore-keys: |
            ${{ runner.os }}-nx-${{ hashFiles('**/package-lock.json') }}-
            ${{ runner.os }}-nx-

      # Run affected tasks in parallel (Nx handles dependency ordering)
      - name: Run affected lint
        run: npx nx affected -t lint --parallel=4 --base=$NX_BASE --
head=$NX_HEAD

      - name: Run affected tests
        run: |
          npx nx affected -t test \
            --parallel=4 \
            --base=$NX_BASE \
            --head=$NX_HEAD \
            --ci \
            --coverage \
            --coverageReporters=lcov

      - name: Run affected builds
        run: npx nx affected -t build --parallel=4 --base=$NX_BASE --
head=$NX_HEAD

      - name: Collect coverage
        uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: coverage
          path: coverage/**/*.lcov

  # Nx affected for infrastructure targets
  infrastructure:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0

      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: '20'
```

```
          cache: 'npm'
      - run: npm ci

      - uses: nrwl/nx-set-shas@v4

      # Run terraform plan for affected infrastructure
      - name: Plan affected infrastructure
        run: |
          npx nx affected -t terraform-plan \
            --base=$NX_BASE \
            --head=$NX_HEAD \
            --parallel=2
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
```

Bazel Integration

Bazel is Google's build system, used for large polyglot monorepos. It provides hermetic, reproducible builds with a fully explicit dependency graph.

```
name: Bazel CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  # Bazel remote cache (Google Cloud Storage, self-hosted, or Buildkite)
  BAZEL_REMOTE_CACHE: "grpc://bazel-cache.internal.my-org.com:9092"

jobs:
  bazel-build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0

      # Mount Bazel cache
      - name: Cache Bazel artifacts
        uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
        with:
          path: |
            ~/.cache/bazel
            ~/.cache/bazelisk
          key: ${{ runner.os }}-bazel-${{ hashFiles('.bazelversion',
'WORKSPACE', 'WORKSPACE.bazel', 'MODULE.bazel') }}
```

```
          restore-keys: |
            ${{ runner.os }}-bazel-

      # Install Bazelisk (Bazel version manager)
      - name: Install Bazelisk
        run: |
          curl -sL
https://github.com/bazelbuild/bazelisk/releases/latest/download/bazelisk-
linux-amd64 \
            -o /usr/local/bin/bazel
          chmod +x /usr/local/bin/bazel
          bazel version

      # Compute affected targets using query
      - name: Compute affected targets
        id: affected
        run: |
          # Get changed files
          if [ "${{ github.event_name }}" = "pull_request" ]; then
            BASE="${{ github.event.pull_request.base.sha }}"
          else
            BASE="${{ github.event.before }}"
          fi

          CHANGED=$(git diff --name-only "$BASE" HEAD | \
            grep -v '^\.github/' || true)

          if [ -z "$CHANGED" ]; then
            echo "No source files changed"
            echo "targets=" >> $GITHUB_OUTPUT
            exit 0
          fi

          # Convert file paths to Bazel labels
          FILE_LABELS=""
          while IFS= read -r FILE; do
            DIR=$(dirname "$FILE")
            # Find the closest BUILD file
            while [ "$DIR" != "." ] && [ ! -f "$DIR/BUILD" ] && [ ! -f
"$DIR/BUILD.bazel" ]; do
              DIR=$(dirname "$DIR")
            done
            if [ -f "$DIR/BUILD" ] || [ -f "$DIR/BUILD.bazel" ]; then
              FILE_LABELS="${FILE_LABELS}//${DIR}:all "
            fi
          done <<< "$CHANGED"

          if [ -z "$FILE_LABELS" ]; then
            echo "targets=" >> $GITHUB_OUTPUT
            exit 0
          fi

          # Expand to all rdeps (reverse dependencies -- everything that
depends on changed targets)
```

```
          AFFECTED=$(bazel query \
            "rdeps(//..., set(${FILE_LABELS}))" \
            --output=label 2>/dev/null | \
            grep -v "@" | \
            head -500 || echo "")    # Limit to 500 targets

          echo "Affected targets:"
          echo "$AFFECTED"

          TARGETS=$(echo "$AFFECTED" | tr '\n' ' ')
          echo "targets=${TARGETS}" >> $GITHUB_OUTPUT

      - name: Build affected targets
        if: steps.affected.outputs.targets != ''
        run: |
          bazel build \
            --remote_cache=${{ env.BAZEL_REMOTE_CACHE }} \
            --remote_upload_local_results=true \
            ${{ steps.affected.outputs.targets }}

      - name: Test affected targets
        if: steps.affected.outputs.targets != ''
        run: |
          bazel test \
            --remote_cache=${{ env.BAZEL_REMOTE_CACHE }} \
            --remote_upload_local_results=true \
            --test_output=errors \
            ${{ steps.affected.outputs.targets }}

      - name: Upload test results
        if: always()
        uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: bazel-test-results
          path: bazel-testlogs/
          retention-days: 7
```

Comparison: Tool Selection Guide

```
TURBOREPO:
  Languages:    JavaScript / TypeScript only
  Graph:        Package-level (npm workspaces)
  Cache:        Content-addressed, remote via Vercel or self-hosted
  GHA fit:      Excellent -- first-class GitHub Actions support
  Learning:     Low
  Best for:     JS/TS fullstack monorepos (Next.js apps + packages)

NX:
  Languages:    JS/TS primary; plugins for Java, Python, Go, .NET
  Graph:        Project-level; more granular than Turborepo
```

```
    Cache:        Nx Cloud (paid) or self-hosted
    GHA fit:      Excellent -- nrwl/nx-set-shas handles SHAs correctly
    Learning:     Medium
    Best for:     Polyglot enterprise monorepos with mixed teams

  BAZEL:
    Languages:    All (hermetic builds for any language)
    Graph:        Target-level (most granular)
    Cache:        Remote cache via gRPC (GCS, S3, self-hosted)
    GHA fit:      Good but complex -- manual affected computation
    Learning:     High (Starlark, BUILD files, hermetic setup)
    Best for:     Large orgs (Google-style) where hermeticity is critical

  WITHOUT BUILD TOOL (custom scripts):
    Languages:    Any
    Graph:        You implement it
    Cache:        actions/cache per service
    GHA fit:      Maximum flexibility, maximum maintenance
    Learning:     Medium-High (you write all the logic)
    Best for:     Simple monorepos, heterogeneous services with no shared
  build graph
```

Interview Answer (30-45 seconds)

> "Monorepo build tools like Turborepo, Nx, and Bazel solve three problems: dependency graph awareness (knowing that changing package A requires rebuilding packages B and C that depend on it), task ordering (running builds in the correct topological order), and build caching (skipping tasks whose inputs haven't changed since the last run). In GitHub Actions, the integration pattern is: let the tool determine what's affected (Turbo's `--filter='...[origin/main]'`, Nx's `affected` command with `nrwl/nx-set-shas` for correct base SHA), back the tool's local cache with `actions/cache` for persistence between runs, and optionally connect to a remote cache (Turbo Remote Cache, Nx Cloud, Bazel remote cache) for cache sharing across branches and machines. The result: a 50-package monorepo CI run that normally takes 30 minutes completes in 2-4 minutes because only 3 packages actually changed and their build outputs were already cached from a previous run."

Gotchas ⚠

- **Turborepo's `--filter='...[origin/main]'` requires the remote `origin/main` to be fetched.** With `fetch-depth: 1`, the comparison target may not exist. Either use `fetch-depth: 0` or explicitly fetch `git fetch origin main` before running turbo.
- **`nrwl/nx-set-shas` sets `NX_BASE` to the last SUCCESSFUL CI run SHA, not HEAD~1.** This is crucial: if commits 1, 2, and 3 land but commit 2's CI failed, `NX_BASE` correctly points to commit 1's SHA so commit 3's CI re-runs anything changed since commit 1 — including commit 2's changes. Using HEAD~1 naively misses this.
- **Bazel hermetic builds fail if build rules reference system tools not in the build container.** Bazel's hermeticity guarantee means your workflow runner's installed tools are irrelevant — Bazel downloads

its own toolchains. This is a feature but surprises teams migrating from Makefile-based builds.

- **Remote cache poisoning in Bazel.** If your remote cache is shared and writable by all, a compromised build can upload malicious cached outputs that other builds consume. Use a read-only cache for PRs and a writable cache only for main branch builds.
- **Turborepo `--dry-run=json` output format changes between versions.** If you parse Turbo's JSON output to extract affected packages, pin the Turbo version and test the parsing when upgrading.

8.5 Cross-Service Dependencies in Monorepos

What It Is

Cross-service dependencies are the relationships between components in a monorepo that must be captured in CI — a change to a shared library should trigger tests for all services that consume it, and a shared interface change should trigger integration tests across all service pairs that communicate through that interface.

This is the hardest operational problem in monorepo CI/CD, and the area most commonly handled incorrectly.

Taxonomy of Dependencies

```
Type 1: Build-time / compile-time dependencies
  packages/shared-types -> used by services/auth, services/payment
  packages/proto -> generates code consumed by services/*
  Change to shared-types: ALL dependent services need recompile + test

Type 2: Runtime API dependencies
  services/payment -> calls services/auth for token validation
  services/orders  -> calls services/inventory for stock checks
  Change to auth's API contract: payment's integration tests may break
  (Even if auth's unit tests pass)

Type 3: Data dependencies
  services/orders writes to orders_db schema
  services/notifications reads orders_db for event triggers
  Schema change: both services affected

Type 4: Infrastructure dependencies
  All services depend on shared/terraform/vpc module
  All services use shared/k8s/base manifests
  Change to VPC module: potentially all service deployments affected
```

Complete Dependency-Aware CI Pipeline

```
name: Cross-Service Dependency CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # ── Stage 1: Detect with full dependency graph ────────────────
  dependency-aware-detection:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.compute.outputs.matrix }}
      has-changes: ${{ steps.compute.outputs.has-changes }}
      run-integration: ${{ steps.compute.outputs.run-integration }}
      integration-pairs: ${{ steps.compute.outputs.integration-pairs }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0

      - name: Compute dependency-aware affected set
        id: compute
        run: |
          # Load the full dependency graph
          # dependency-graph.json structure:
          # {
          #   "services": {
          #     "auth":      { "deps": ["shared/types", "shared/lib"] },
          #     "payment":   { "deps": ["shared/types", "shared/proto",
"shared/lib"] },
          #     "inventory": { "deps": ["shared/types", "shared/lib"] },
          #     "orders":    { "deps": ["shared/types", "shared/proto",
"shared/lib"] }
          #   },
          #   "runtime_deps": {
          #     "payment":   ["auth"],
          #     "orders":    ["inventory", "auth"],
          #     "notifications": ["orders"]
          #   },
          #   "shared_packages": ["shared/types", "shared/lib",
"shared/proto"]
          # }

          python3 << 'PYEOF'
          import subprocess, json, os, sys
          from collections import defaultdict, deque

          # Get changed files
```

```
            event = os.environ.get('GITHUB_EVENT_NAME', 'push')
            if event == 'pull_request':
                base = os.environ.get('BASE_SHA', 'HEAD~1')
                head = os.environ.get('HEAD_SHA', 'HEAD')
            else:
                before = os.environ.get('BEFORE_SHA', '')
                base = before if before and before != '0' * 40 else 'HEAD~1'
                head = 'HEAD'

            result = subprocess.run(
                ['git', 'diff', '--name-only', base, head],
                capture_output=True, text=True
            )
            changed_files = set(result.stdout.strip().split('\n')) if
  result.stdout.strip() else set()
            print(f"Changed: {changed_files}", flush=True)

            # Load graph
            with open('dependency-graph.json') as f:
                graph = json.load(f)

            services = graph['services']
            runtime_deps = graph.get('runtime_deps', {})
            shared_pkgs = graph.get('shared_packages', [])

            def files_touch_path(files, path_prefix):
                return any(f.startswith(path_prefix + '/') for f in files)

            # Step 1: Direct compile-time affected
            compile_affected = set()
            for svc, config in services.items():
                # Direct service change
                if files_touch_path(changed_files, f'services/{svc}'):
                    compile_affected.add(svc)
                    continue
                # Shared dependency change
                for dep in config.get('deps', []):
                    if files_touch_path(changed_files, dep):
                        compile_affected.add(svc)
                        break

            print(f"Compile-time affected: {compile_affected}", flush=True)

            # Step 2: Runtime API affected (transitive)
            # Build reverse runtime dep graph
            reverse_runtime = defaultdict(set)
            for consumer, producers in runtime_deps.items():
                for producer in producers:
                    reverse_runtime[producer].add(consumer)

            # BFS to find all transitively affected services
            runtime_affected = set()
            queue = deque(compile_affected)
            while queue:
```

```
                svc = queue.popleft()
                if svc in runtime_affected:
                    continue
                runtime_affected.add(svc)
                # Add all services that call this service
                for dependent in reverse_runtime.get(svc, set()):
                    if dependent not in runtime_affected:
                        queue.append(dependent)

            all_affected = compile_affected | runtime_affected
            print(f"All affected (including transitive): {all_affected}",
flush=True)

            # Step 3: Identify integration test pairs needed
            # An integration test pair (A, B) is needed when:
            # - A calls B and A or B is affected
            integration_pairs = []
            for consumer, producers in runtime_deps.items():
                for producer in producers:
                    if consumer in all_affected or producer in all_affected:
                        pair = {'consumer': consumer, 'producer': producer}
                        if pair not in integration_pairs:
                            integration_pairs.append(pair)

            run_integration = len(integration_pairs) > 0
            print(f"Integration test pairs: {integration_pairs}",
flush=True)

            # Build matrix
            matrix_items = [
                {'service': svc, 'type': 'compile-affected' if svc in
compile_affected else 'runtime-affected'}
                for svc in sorted(all_affected)
            ]
            matrix = {'include': matrix_items}
            has_changes = len(all_affected) > 0

            # Write outputs
            with open(os.environ['GITHUB_OUTPUT'], 'a') as f:
                f.write(f"matrix={json.dumps(matrix, separators=(',',
':'))}\n")
                f.write(f"has-changes={'true' if has_changes else
'false'}\n")
                f.write(f"run-integration={'true' if run_integration else
'false'}\n")
                f.write(f"integration-pairs={json.dumps(integration_pairs,
separators=(',', ':'))}\n")
            PYEOF
          env:
            BASE_SHA: ${{ github.event.pull_request.base.sha }}
            HEAD_SHA: ${{ github.event.pull_request.head.sha }}
            BEFORE_SHA: ${{ github.event.before }}

  # ── Stage 2: Unit tests for directly affected services ──────────
```

```yaml
  unit-tests:
    needs: dependency-aware-detection
    if: needs.dependency-aware-detection.outputs.has-changes == 'true'
    strategy:
      fail-fast: false
      matrix: ${{ fromJSON(needs.dependency-aware-
detection.outputs.matrix) }}
    runs-on: ubuntu-latest
    name: "unit-test: ${{ matrix.service }} (${{ matrix.type }})"
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Run unit tests
        working-directory: services/${{ matrix.service }}
        run: make test-unit

  # ── Stage 3: Integration tests for affected pairs ────────────────
  integration-tests:
    needs: [dependency-aware-detection, unit-tests]
    if: |
      needs.dependency-aware-detection.outputs.run-integration == 'true'
&&
      needs.unit-tests.result == 'success'
    strategy:
      fail-fast: false
      matrix: ${{ fromJSON(needs.dependency-aware-
detection.outputs.integration-pairs) }}
    runs-on: ubuntu-latest
    name: "integration: ${{ matrix.consumer }} -> ${{ matrix.producer }}"
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      # Start both services for integration test
      - name: Start ${{ matrix.producer }} service
        run: |
          docker run -d \
            --name ${{ matrix.producer }} \
            --network ci-test-net \
            ghcr.io/my-org/${{ matrix.producer }}:sha-${{ github.sha }}

      - name: Start ${{ matrix.consumer }} service
        run: |
          docker run -d \
            --name ${{ matrix.consumer }} \
            --network ci-test-net \
            --env PRODUCER_URL=http://${{ matrix.producer }}:8080 \
            ghcr.io/my-org/${{ matrix.consumer }}:sha-${{ github.sha }}

      - name: Run integration tests
        run: |
          cd integration-tests/${{ matrix.consumer }}-${{ matrix.producer
}}/
          make test

      - name: Collect logs on failure
```

```
        if: failure()
        run: |
          docker logs ${{ matrix.producer }} || true
          docker logs ${{ matrix.consumer }} || true
```

Shared Package Change — Full Rebuild Strategy

```
# When a foundational shared package changes, you may need to rebuild ALL
services
# This is the "nuclear option" that ensures correctness at the cost of CI
time

  detect-shared-changes:
    runs-on: ubuntu-latest
    outputs:
      rebuild-all: ${{ steps.check.outputs.rebuild-all }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0

      - uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
        id: filter
        with:
          filters: |
            critical-shared:
              - 'packages/api-types/**'      # API contracts
              - 'packages/database/**'       # Database schemas
              - 'packages/authentication/**' # Auth primitives

      - name: Check if full rebuild needed
        id: check
        run: |
          if [ "${{ steps.filter.outputs.critical-shared }}" = "true" ];
then
            echo "Critical shared package changed -- triggering full
rebuild"
            echo "rebuild-all=true" >> $GITHUB_OUTPUT
          else
            echo "rebuild-all=false" >> $GITHUB_OUTPUT
          fi

  # Build ALL services if critical shared changed
  full-rebuild:
    needs: detect-shared-changes
    if: needs.detect-shared-changes.outputs.rebuild-all == 'true'
    strategy:
      fail-fast: false
      matrix:
        service: [auth, payment, inventory, orders, notifications, api-
```

```
gateway]
    runs-on: ubuntu-latest
    name: "rebuild-all: ${{ matrix.service }}"
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make build test SERVICE=${{ matrix.service }}
```

Managing Proto Changes (API Contract Evolution)

```
# Proto changes are one of the most common sources of cross-service
breakage
# Pattern: detect proto change -> regenerate code -> test all consumers

name: Proto CI

on:
  push:
    paths:
      - 'proto/**'
      - '.github/workflows/proto.yml'
  pull_request:
    paths:
      - 'proto/**'

jobs:
  check-breaking-changes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0

      - name: Install buf
        run: |
          BIN="/usr/local/bin"
          curl -sSL
"https://github.com/bufbuild/buf/releases/latest/download/buf-Linux-
x86_64" \
            -o "${BIN}/buf"
          chmod +x "${BIN}/buf"

      - name: Check for breaking proto changes
        run: |
          # buf detects backward-incompatible proto changes
          buf breaking \
            --against '.git#branch=main' \
            proto/

      - name: Lint proto files
        run: buf lint proto/
```

```yaml
  regenerate-and-test:
    needs: check-breaking-changes
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Generate code from protos
        run: |
          buf generate proto/
          # Generated: services/*/generated/

      - name: Test all services with new generated code
        strategy:
          matrix:
            service: [auth, payment, orders]  # Services that use protos
        uses: ./.github/workflows/service-test.yml@main
        with:
          service: ${{ matrix.service }}
```

Schema Migration Safety Check

```yaml
# Database schema changes require cross-service compatibility check
# Pattern: run schema migration, then test all services that read from the
schema

  schema-compat-check:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_PASSWORD: testpass
          POSTGRES_DB: testdb
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Apply new migrations
        run: |
          ./flyway migrate \
            -url=jdbc:postgresql://localhost:5432/testdb \
            -user=postgres \
```

```
            -password=testpass

      - name: Test services against new schema
        run: |
          # Run read-path tests for all services that read from this
schema
          for SERVICE in orders notifications analytics; do
            echo "Testing $SERVICE against new schema"

DATABASE_URL="postgres://postgres:testpass@localhost:5432/testdb" \
              make test-schema-compat SERVICE=$SERVICE
          done
```

Interview Answer (30-45 seconds)

> "Cross-service dependencies are the hardest part of monorepo CI. There are four dependency types: compile-time (shared libraries), runtime API (service A calls service B), data (shared database schema), and infrastructure. For compile-time deps, the dependency graph maps each service to its shared package dependencies — when a shared package changes, all consumers are added to the affected set. For runtime API deps, you need transitive closure: if payment calls auth and auth changes, payment's integration tests must run too. The implementation uses BFS on the reverse runtime dependency graph starting from the compile-time affected set. For proto/API contract changes specifically, `buf breaking` catches backward-incompatible changes before code generation. The key principle: when in doubt, over-trigger. A false negative (missing an affected service) causes a production incident; a false positive (running an unnecessary test) just wastes 5 minutes of CI time."

Gotchas ⚠

- **Circular runtime dependencies produce infinite BFS loops.** Services should have a clean DAG for API calls. If service A calls B and B calls A, either there's an architectural problem, or you need to mark one direction as an event/queue relationship (not synchronous API) and exclude it from the dep graph.
- **Transitive affected set grows large for highly connected graphs.** If `shared/types` is imported by every service, a one-line change to `shared/types` makes all 50 services "affected." Consider tiered shared packages: `shared/core` (stable, rarely changes) vs `shared/utils` (frequently updated, lightweight). Changes to core trigger full rebuilds; changes to utils use the normal affected detection.
- **Integration tests between services require compatible image versions.** If you're building new images in the same workflow run and using them for integration tests, the new image must be pushed to the registry before the integration test job pulls it. Ensure the build job completes before the integration test job starts (via `needs:`).
- **Proto backward compatibility checking must be against the committed proto, not the working tree.** `buf breaking --against '.git#branch=main'` compares against the mainline committed proto. If your main branch has uncommitted local changes, the comparison is wrong. Always test against a clean branch ref.

- **Schema migrations in CI must be isolated from production.** Use a fresh ephemeral database (service container) for migration testing, never a shared dev database. A failed migration test must not affect other teams' work.

Cross-Topic Interview Questions

Q: You have a monorepo with 40 services. CI takes 45 minutes and nobody understands why. Walk through how you'd diagnose and fix it.

> "First, get visibility: use the GitHub Actions timing breakdown and look at the workflow run graph to find the critical path. Typically the problem is one of four things.
>
> One: no path filtering. Every commit to the docs folder triggers all 40 service CIs. Fix: path filters per service workflow, or `dorny/paths-filter`-based dynamic matrix.
>
> Two: no caching. `npm ci` is downloading 400MB of packages every run. Fix: `actions/cache` with `hashFiles(package-lock.json)` key, or the built-in `actions/setup-node cache: npm`.
>
> Three: no parallelism or wrong parallelism. Services that should run in parallel are in a chain. Fix: dynamic matrix with `fail-fast: false`. If parallelism exists, check `max-parallel` isn't artificially limiting it.
>
> Four: build tool not being used effectively. If it's a JS monorepo and you're running `npm test --workspaces` (builds everything), switching to `turbo test --filter='...[origin/main]'` can take a 45-minute run to 4 minutes for typical PRs.
>
> After fixing, set up metrics: graph the 90th percentile CI time per service per week. If any service starts trending up, catch it before it becomes the new bottleneck."

Q: A shared package changed that's used by 30 of your 40 services. How do you handle this in your dynamic matrix CI?

> "Three strategies depending on the package. If it's a truly foundational package like the ORM models or API types, the correct answer is full rebuild of all 30 consumers — add a `full-rebuild` job triggered when `critical-shared` path filter matches, using a static matrix of all consumer services. The correctness requirement outweighs the CI time cost.
>
> If it's a utility package with low blast radius, use the dependency graph: only rebuild services whose `deps` array includes the changed package, not all 30 automatically. This requires maintaining an accurate `dependency-graph.json`.
>
> If this pattern happens frequently and causes pain, the architectural fix is better package layering: split shared packages into `core` (stable, changes monthly) vs `utils` (updated frequently, minimal interface). Changes to `core` trigger full rebuilds; changes to `utils` only affect direct importers. The goal is that the typical PR touches only one package and its 2-3 direct consumers, not all 30."

Q: How do you ensure a PR that only changes README.md doesn't block on a 30-minute CI requirement?

> "This is the path filter vs branch protection conflict. Three approaches in order of recommendation.
>
> One: use `paths-ignore` instead of `paths` on all service workflows. The workflow runs for everything EXCEPT pure docs changes. For a README-only PR, CI runs but takes only 2 minutes because no source changed (fast lint/type-check passes, no builds triggered).
>
> Two: implement the `ci-gate` pattern. A final job `if: always()` checks all upstream job results and posts a green status check if upstream jobs either succeeded OR were appropriately skipped. Branch protection requires `ci-gate` not the individual service tests. This way a README-only PR gets a green `ci-gate` status without waiting for any service builds.
>
> Three: use GitHub's 'Allowed to skip' feature on branch protection -- any workflow run that was skipped due to path filters is treated as a pass for the purposes of the required check. This is a GitHub Enterprise feature.
>
> I'd implement option two as the standard pattern because it also handles the case where you add a new service and forget to add its status check to branch protection -- the gate adapts automatically."

Quick Reference Cheat Sheet

Path Filter Gotchas

```
# GOOD: Include workflow file in its own paths list
on:
  push:
    paths:
      - 'services/payment/**'
      - '.github/workflows/payment.yml'   # <-- REQUIRED

# GOOD: Use ** for recursive matching
paths:
  - 'services/payment/**'   # matches all files at any depth

# BAD: paths and paths-ignore on same event
on:
  push:
    paths: ['services/**']
    paths-ignore: ['**/*.md']   # ERROR -- choose one
```

Detection: paths-filter vs Custom Script

```
dorny/paths-filter:
  + Simple config (YAML filter definitions)
  + outputs.changes = JSON array of matched filter names
  + PR-aware (correct diff computation for forks)
  Use when: < 20 services, no transitive dep tracking needed

Custom git diff:
  + Full control over dependency graph
  + Transitive closure of runtime API dependencies
  + Service-specific metadata in matrix entries
  Use when: complex dep graph, runtime API dependencies matter
```

Dynamic Matrix Recipe

```yaml
# Step 1: Output matrix from detect job
- id: detect
  run: echo "matrix={...}" >> $GITHUB_OUTPUT

# Step 2: Use in downstream job
needs: detect
if: needs.detect.outputs.has-changes == 'true'   # Guard for empty matrix
strategy:
  fail-fast: false
  matrix: ${{ fromJSON(needs.detect.outputs.matrix) }}
runs-on: ${{ matrix.runner }}      # Per-service runner
timeout-minutes: ${{ fromJSON(matrix.timeout_minutes) }}
name: "${{ matrix.service }}"

# Step 3: Always-running gate job
needs: [detect, build-test, deploy]
if: always()
# -> Satisfies branch protection even for PRs that skip all jobs
```

Build Tool Quick Reference

```
Turborepo:   JS/TS only
  npx turbo <task> --filter='...[origin/main]'
  Cache: .turbo/ dir + actions/cache
  SHA: origin/main ref (fetch-depth: 0 required)

Nx:          Polyglot
  npx nx affected -t <task> --base=$NX_BASE --head=$NX_HEAD
  Cache: ~/.nx/cache + actions/cache
  SHA: nrwl/nx-set-shas action (uses last successful CI run)

Bazel:       All languages, hermetic
  bazel test $(bazel query "rdeps(//..., set($CHANGED_LABELS))")
```

```
  Cache: ~/.cache/bazel + remote gRPC cache
  SHA: manual git diff + bazel query for label mapping
```

Dependency Type Decision

```
Change to shared library     -> All consumers: compile-time affected
                                  (direct rebuild)
Change to service API        -> All callers: runtime-affected
                                  (integration tests)
Change to proto/schema       -> Regenerate + test all consumers
                                  (buf breaking check first)
Change to DB schema          -> Test all services reading that schema
                                  (ephemeral DB in service container)
Change to infra (terraform)  -> Terraform plan, NOT service tests
                                  (separate infra workflow)
```

CI Gate Pattern (Satisfies Branch Protection for Skipped Jobs)

```
ci-gate:
  needs: [detect, build-test, deploy]
  if: always()
  runs-on: ubuntu-latest
  steps:
    - run: |
        for RESULT in \
          "${{ needs.detect.result }}" \
          "${{ needs.build-test.result }}" \
          "${{ needs.deploy.result }}"; do
          [ "$RESULT" = "failure" ] || [ "$RESULT" = "cancelled" ] && exit
1
        done
        echo "Gate passed"
# Branch protection requires: ci-gate (not individual service jobs)
# README-only PR: detect=success, build-test=skipped -> ci-gate=success
```

End of Category 8: Monorepo Patterns

> **Next:** Category 9 (Observability, Debugging & Governance) covers the operational questions you'll face after deploying this monorepo CI system at scale: structured logging, workflow analytics, rate limits, and audit trails — plus the debugging tools (tmate, act, step summaries) needed when these complex workflows inevitably misbehave.
>
> **Priority drill topics from this category:**

> - Dynamic matrix + empty guard: write the full two-job pattern from memory including the has-changes guard, `fromJSON()` call, and `fail-fast: false`
> - Path filter + branch protection: explain the conflict and the three solutions (paths-ignore, ci-gate, GHE skip)
> - Transitive dependency closure: explain BFS from compile-time affected set using reverse runtime dep graph
> - Nx `nrwl/nx-set-shas`: explain WHY it uses the last successful run SHA rather than HEAD~1